

图像处理基础课程课后作业八

建智 2201 谢天赐 20221082044

8.1 完成课堂练习

通过课上学习课下练习，按要求完成代码如下：

```
# 确保图像读取成功
if src is None:
    print("图像读取失败，请检查文件路径。")
else:
    # 获得图像尺寸和阈值
    h, w = src.shape[:2]
    thrd = 127

    # 设置结果（边界）图像数组
    rst = np.zeros((h, w), np.uint8) + 255

    # 定义扫描周围像素点的偏移量
    m, n = [0, 1, 0, -1], [-1, 0, 1, 0]

    # 扫描图像全部像素点（行、列）
    for i in range(h):
        for j in range(w):
            k1, k2 = 0, 0 # 前景点k1、背景点k2计数器置于0
            for k in range(4): # 扫描当前像素点周围4个像素点
                ii, jj = i + m[k], j + n[k]
                if ii >= 0 and jj >= 0 and ii < h and jj < w: # 判断周边第k个点是否在图像范围内
                    t = src[ii, jj]
                    if t >= thrd:
                        k1 += 1
                    else:
                        k2 += 1
            # 由前景/背景点总数判定是否为边界点
            if k1 >= 1 and k2 >= 1:
                rst[i, j] = 0 # 如果为边界点，则对结果图像赋值
```

图 1

经过边缘检测处理后的图 8. 1a、8. 1b 图形边界效果如图 2、图 3 所示：

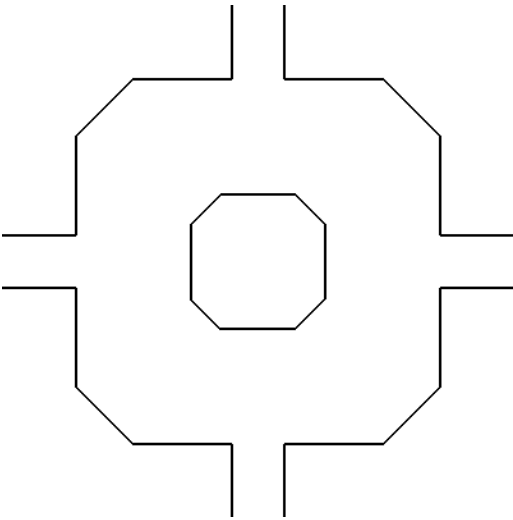


图 2

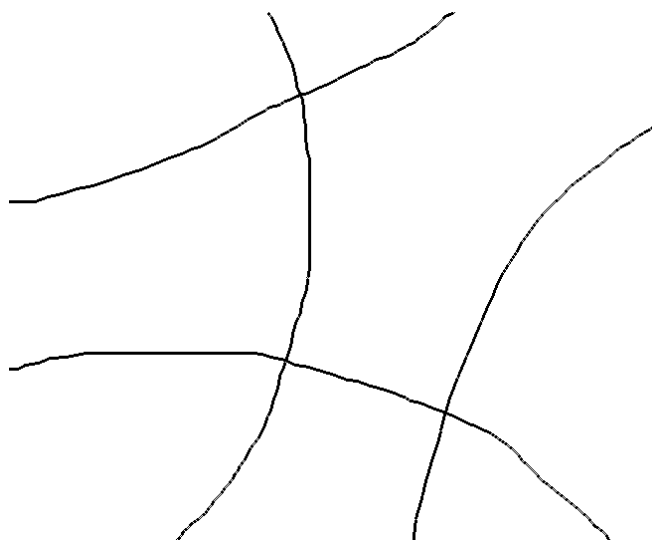


图 3

8.2 研究分析

问题 1:

比较十字形和米字型扫描图像的区别。

通过修改偏移量的定义以及扫描像素点的循环次数，我将十字形（四个点）的处理方式改为了米字型（八个点）分别得到结果为图 4、5 所示，左图为十字形，右图为米字型。

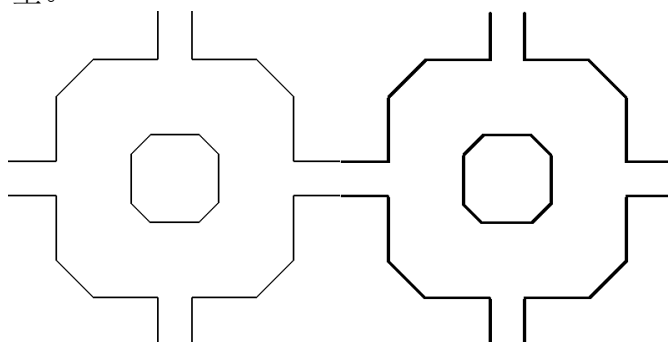


图 4



图 5

在图片结果的对比上，显而易见的是米字型扫描方式得到的边缘图形像素线条要粗与十字形方式。

经过对其原理和代码更为细致的分析得出，使用四个点和八个点进行边缘检

测扫描的主要区别在于检测到的边缘的精细度和对噪声的敏感度。

以下是两种方法的具体区别：

边缘精细度：

四个点（4-邻域）：这种方法只考虑像素的直接上下左右邻居，因此检测到的边缘可能比较粗糙，因为它只捕捉到水平和垂直方向上的边缘变化。

八个点（8-邻域）：这种方法考虑像素的所有直接邻居（包括对角线方向），因此可以捕捉到更多细节，包括对角线方向上的边缘变化，从而得到更精细的边缘检测结果。

对噪声的敏感度：

四个点：由于只考虑四个邻居，这种方法对噪声的敏感度相对较低。小的噪声点不太可能同时影响四个邻居，因此可能不会导致错误的边缘检测。

八个点：由于考虑了更多的邻居，这种方法对噪声更敏感。一个小的噪声点可能会影响多个邻居，从而增加误检边缘的风险。

计算复杂度：

四个点：计算量较小，因为每个像素点只需要检查四个邻居。

八个点：计算量较大，因为每个像素点需要检查八个邻居。

边缘连续性：

四个点：可能无法很好地检测到边缘的连续性，特别是在边缘方向不完全水平或垂直的情况下。

八个点：由于考虑了对角线方向，通常能更好地检测边缘的连续性。

问题 2：

8.1 中可能存在的问题：

```
for i in range(h):
    for j in range(w):
        k1, k2 = 0, 0 # 前景点k1、背景点k2计数器置于0
        for k in range(4): # 扫描当前像素点周围4个像素点
            ii, jj = i + m[k], j + n[k]
            if ii >= 0 and jj >= 0 and ii < h and jj < w: # 判断周边第k个点是否在图像范围内
                t = src[ii, jj]
                if t >= thrd:
                    k1 += 1
                else:
                    k2 += 1
```

对于边缘的像素点，它们没有足够的邻居点，因为它们的一部分邻居会超出图像的边界。

例如，位于图像最左边或最右边的像素点没有左边或右边的邻居，位于图像最上面或最下面的像素点没有上面的邻居。如果我们不限制循环的范围，那么在处理边缘像素时，代码会尝试访问这些不存在的邻居点，从而导致索引越界错误。

最左边和最右边的像素点：对于最左边的像素点，它们的左边没有邻居；对于最右边的像素点，它们的右边没有邻居。

最上面和最下面的像素点：对于最上面的像素点，它们上面没有邻居；对于最下面的像素点，它们下面没有邻居。

角落的像素点：位于图像四个角落的像素点没有足够的邻居点。

所以，我做了以下修改：

```

18 # 扫描图像全部像素点（行、列）
19 for i in range(1, h-1): # 减1是因为边界像素没有足够的邻居点
20     for j in range(1, w-1): # 减1是因为边界像素没有足够的邻居点
21         k1, k2 = 0, 0 # 前景点k1、背景点k2计数器置于0
22         for k in range(8): # 扫描当前像素点周围8个像素点
23             ii, jj = i + m[k], j + n[k]
24             if ii >= 0 and jj >= 0 and ii < h and jj < w: # 判断周边第k个点是否在图像范围内
25                 t = src[ii, jj]
26                 if t >= thrd:
27                     k1 += 1
28                 else:
29                     k2 += 1
30             # 由前景/背景点总数判定是否为边界点
31             if k1 >= 1 and k2 >= 1:
32                 rst[i, j] = 0 # 如果为边界点，则对结果图像赋值
33

```

通过将循环的范围限制为`range(1, h-1)`和`range(1, w-1)`们确保了在处理每个像素点时，它的所有邻居都在图像范围内。这样就可以安全地访问这些邻居点，而不会导致索引越界错误。例如，如果我们有一个 5x5 的图像，循环的范围应该是：这样就可以安全地处理每个像素点及其邻居，而不会访问超出图像范围的像素点。

在扫描八个点的代码中还需要确保循环的范围不包括图像的边缘像素，因为这些像素没有足够的邻居点。

问题 3:

为了能够使结论具有普遍性，实验主要选择了复杂的简笔图像以及真实的复杂图像。复杂简笔图像如图 6 所示，真实复杂图像为图 7 所示。



图 6

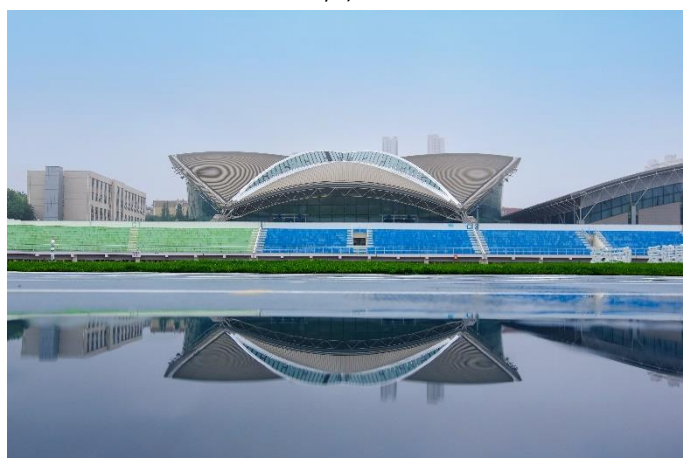


图 7

分别使用了十字形以及米字型的扫描方式，结果为图 8 所示，可以看出目前这种边缘扫描方式在稍微复杂的简笔图像中仍然具有较好的效果，同时也可以注意到计算时间稍长，以及在面对较细小的边缘时存在遗漏的情况

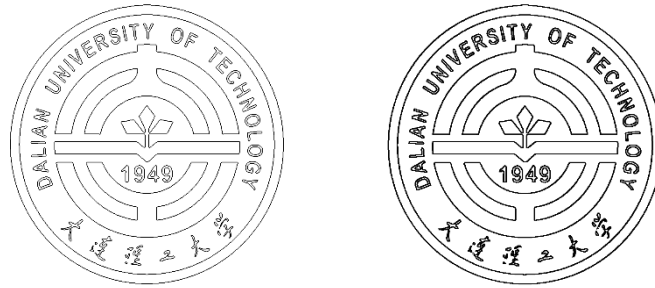


图 8

图 9 可以看出目前这种边缘扫描方式在复杂的真实图像中具有一定效果，然而存在噪点等问题，同时计算时间很长，效率较为低下。

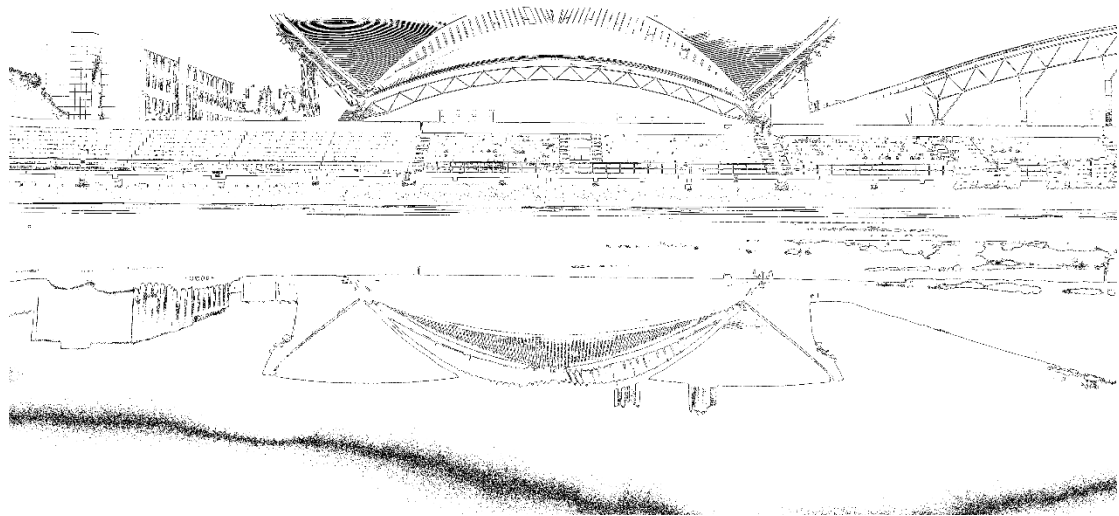


图 9

改进方法：

为了解决计算效率低下以及受噪声影响导致计算效果一般的问题，我们主要使用了 Canny 边缘检测。

Canny 算法中具有一系列方式来提升图片处理效果：

噪声降低：

Canny 算法的第一步是使用**高斯滤波器**对图像进行平滑处理，以减少图像中的噪声。这是因为边缘检测对噪声非常敏感，噪声可能会导致错误的边缘检测。

梯度计算：

接下来，算法计算图像的梯度幅度和方向。这通常是通过在图像上应用 Sobel 算子来完成的，该算子可以分别计算图像在水平和垂直方向上的梯度。梯度幅度是这两个方向梯度的欧几里得范数，而梯度方向是这两个方向梯度的反正切值。

非极大值抑制：

非极大值抑制的目的是细化边缘，确保边缘是单像素宽。这一步骤沿着梯度方向检查每个像素点，只保留梯度幅度局部最大的点，即边缘候选点。

双阈值检测：

为了确定哪些边缘是真实的边缘，Canny 算法使用双阈值策略。算法设置两个阈值，一个高阈值和一个低阈值。高于高阈值的梯度被认为是强边缘，低于低阈值的梯度被认为是非边缘，而在这两个阈值之间的梯度被认为是边缘的候选。

边缘跟踪：

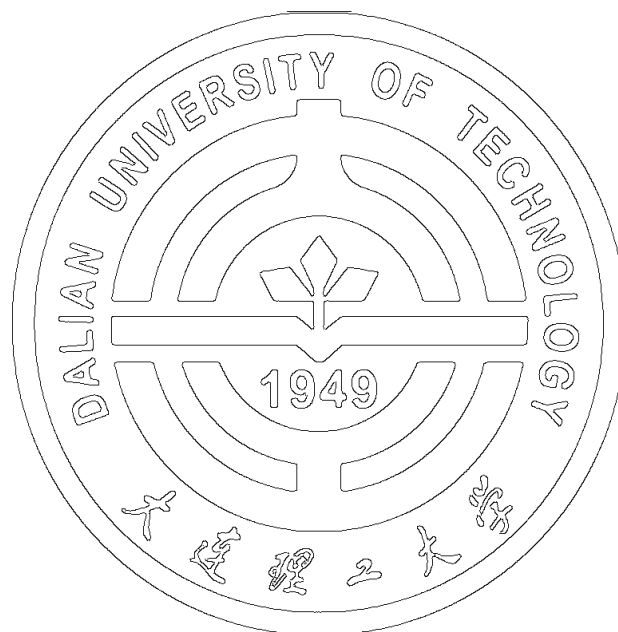
在双阈值检测之后，算法通过边缘跟踪来连接强边缘和弱边缘。如果一个弱边缘与一个强边缘相连，则被认为是真实的边缘的一部分。

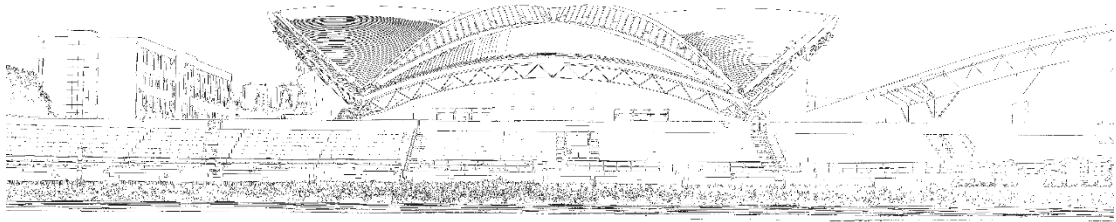
滞后阈值：

滞后阈值是一种实现边缘跟踪的技术。它使用两个阈值（低和高）来确定边缘。如果一个像素的梯度值高于高阈值，它将被标记为边缘。如果一个像素的梯度值低于低阈值，它将被排除。如果梯度值介于这两个阈值之间，它将被标记为边缘，但只有在它连接到一个梯度值高于高阈值的像素时。通过这些步骤，Canny 算法能够有效地检测图像中的边缘，同时对噪声具有较好的抑制作用，并且能够保持边缘的精确性和完整性。

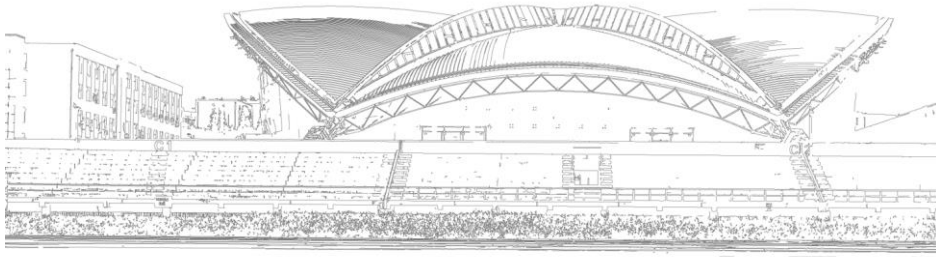
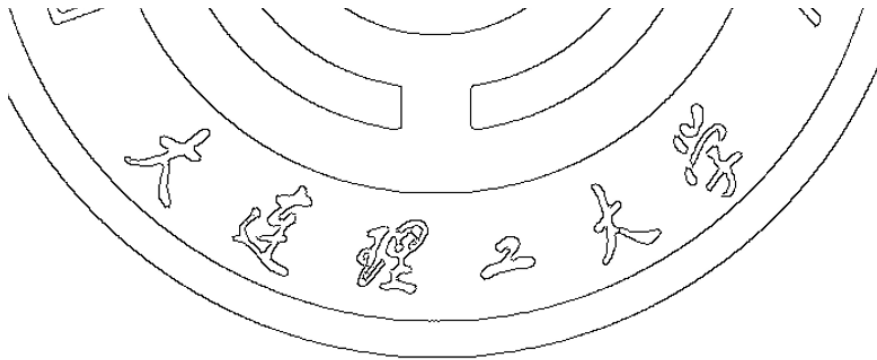
同时，由于 Canny 算法是基于优化过的 C/C++ 语言实现的，具有较好的时间复杂度，效率较高。

得到的结果图如图所示：





局部细节如图所示：



可以看出，Canny 算法在边缘细节处理上具有较好效果。

8.3

卷积的定义：

图像卷积运算是一种数学处理过程，它涉及将图像（信号）与一个称为卷积核（或滤波器）的小矩阵进行运算，以产生一个新的图像（信号）。这个过程可以表示为两个函数的积分或求和，其中一个函数是输入信号，另一个是卷积核。

卷积核：

卷积核是一个小的矩阵，用于定义卷积运算中使用的权重。不同的卷积核可以产生不同的图像效果，如模糊、锐化、边缘检测等。

卷积过程：

在卷积过程中，卷积核被放置在图像的每个像素上，并根据卷积核中的权重与图像的局部区域进行加权求和，从而计算出新图像中对应像素的值。

边界处理：

当卷积核超出图像边界时，需要定义如何处理这些边界情况。常见的边界处理方法包括常数填充、复制边缘像素、镜像等。

线性和非线性滤波：

线性滤波（如均值滤波和高斯滤波）涉及简单的加权求和，而非线性滤波（如中值滤波）则涉及对像素值的排序和选择。

卷积的数学表达：

卷积运算可以用数学公式表示为：

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau)g(t - \tau)d\tau$$

其中 f 是输入信号， g 是卷积核， t 是时间变量。

卷积的效果：

卷积可以用于平滑图像（去除噪声）、增强图像边缘（锐化）、检测特定特征（如边缘）等。

对于二维卷积核，卷积运算可以分解为两次一维卷积运算，这可以减少计算量。